School of Engineering and Applied Science (SEAS), Ahmedabad University
CSE 400: Fundamentals of Probability in Computing

Project Scribe Report: Milestone 2

Probabilistic Modeling & Mathematical Formulation

Name:	Urvil Chodvadiya and Manthan Pancholi
Group Number:	Group 8
Email:	urvil.c@ahduni.edu.in, manthan.p1@ahduni.edu.in
Date of Submission:	February 19, 2026

Scribe Question 1: Project System and Objective

What is the probabilistic problem being addressed in your project? Clearly state the system objective and identify the primary sources of uncertainty involved.

1.1 The Probabilistic Problem

The project addresses the fundamental challenge of Task Offloading in Device-to-Device (D2D) Edge Networks under conditions of stochastic processing times.

In traditional edge computing models, the system assumes that the execution time of a task is deterministic—i.e., if a task T is offloaded to device D, it will strictly take t seconds. However, this assumption fails in real-world Mobile Edge Computing (MEC) scenarios due to the heterogeneous and dynamic nature of mobile devices.

The core probabilistic problem is that **Task Processing Time (TPT)** is a random variable, not a constant.

- When a device offloads a task, it cannot know the exact completion time T_{finish}.
- Instead, it can only estimate the probability $P(T_{finish} \leq \tau_{deadline})$.
- This creates a cascading effect: If one task takes longer than expected (a "tail event" in the probability distribution), it delays all subsequent tasks in the queue, causing a probabilistic violation of deadlines for multiple users.

1.2 System Objective

The objective is not simply to minimize the average latency, which is a common mistake in deterministic systems. Minimizing the average does not guarantee that specific deadlines are met.

Instead, the system objective is to maximize the Global Task Completion Rate. Let Y_i be a Bernoulli random variable such that $Y_i = 1$ if $T_{sojourn,i} \leq \tau_i$ and 0 otherwise. The system objective is:

$$\max_{\mathbf{a}} \sum_{i \in N} \mathbb{E}[Y_i] \tag{1}$$

where $\mathbb{E}[Y_i] = P(T_{sojourn,i} \leq \tau_i)$. We seek to find an optimal offloading strategy vector $\mathbf{a}$ that achieves this.

This requires solving a joint optimization problem where we must balance the computational load across devices to minimize the probability of queue overflow and excessive delays.

1.3 Primary Sources of Uncertainty

The project identifies and models three distinct layers of uncertainty:

1. Intrinsic Processing Uncertainty (The Root Cause): The CPU time required to execute a specific code block varies based on the input data. For example, in a Visual Relationship Detection (VRD) task, processing a blank image is fast, while processing a crowded street scene is slow. This variance is modeled via a Probability Density Function (PDF).

2. Arrival Uncertainty (Traffic Bursts): Tasks are not generated at fixed intervals. User behavior is sporadic. This is modeled as a Stochastic Process (Poisson Arrival), meaning we cannot predict exactly when the next task will arrive, only the rate λ.

3. Queueing Uncertainty (The Derived Effect): Because arrivals and processing times are random, the "Waiting Time" in the buffer is highly volatile. A device might be empty now, but overloaded 10ms later. This requires Queueing Theory (M/G/1) to model.

Scribe Question 2: Key Random Variables and Uncertainty Modeling

Identify the key random variables in your project and describe how uncertainty is modeled for each. Clearly state any probabilistic assumptions currently being made.

2.1 Key Random Variables

To strictly model the system, we define the following random variables (RVs):

- X_{proc} (Task Processing Time): A continuous RV representing service time.
- T_{inter} (Inter-arrival Time): We assume tasks follow a Poisson process, meaning $T_{inter} \sim$ **Exponential**(λ).
- W_q (Queue Waiting Time): A derived RV representing the buffer delay.
- L_q (Queue Length): For our current $M/M/1$ setup, the number of tasks in the buffer follows a **Geometric distribution**.
- B (Bandwidth Demand): A random variable representing the task size in bits, influencing transmission delay.
- H (Channel State): An RV representing network link quality; while assumed static for Milestone 2, it is defined here for stochastic evolution in Milestone 3.

2.1.1 Multivariate Modeling

To capture the interaction between network traffic and device performance, we define the **Joint PDF** $f_{W_q,X_{proc}}(w,x)$. Under the assumption of independence (Section 2.4), this simplifies to $f_{W_q}(w) \cdot f_{X_{proc}}(x)$, but in congested states, these factors become stochastically dependent.

2.2 Mathematical Modeling of Processing Uncertainty

The base paper utilizes empirical data (from VRD applications) to characterize the Task Processing Time. The distribution of processing times is observed to fit an Exponential Distribution.

Probability Density Function (PDF): We model the processing time t for a task on a device with computing capability c as:

$$f(t) = \mu e^{-\mu t}, \quad t \geq 0 \tag{2}$$

where μ is the processing rate parameter.

- μ is derived from the device frequency c and the task workload density.
- The expected (mean) processing time is $E[t] = \frac{1}{\mu}$.
- The variance is $Var(t) = \frac{1}{\mu^2}$. This variance explicitly captures the "uncertainty" in execution.

To verify the expected processing time $E[t] = 1/\mu$, we derive it from the definition of expectation for a continuous random variable:

$$E[t] = \int_0^\infty t \cdot f(t)\, dt = \int_0^\infty t \cdot \mu e^{-\mu t}\, dt$$

Using integration by parts ($u = t, dv = \mu e^{-\mu t}\, dt$):

$$E[t] = [-te^{-\mu t}]_0^\infty + \int_0^\infty e^{-\mu t}\, dt = 0 + \left[-\frac{1}{\mu}e^{-\mu t}\right]_0^\infty = \frac{1}{\mu}$$

This step-by-step integration confirms that our model's rate parameter μ is directly tied to the average CPU cycles required.

2.3 Mathematical Modeling of System Dynamics (M/G/1 Queue)

Since we have Poisson arrivals (Markovian) and Exponential processing times (which is a specific case of General distribution, though the paper generalizes to M/G/1 to allow future flexibility), the edge device is modeled as a queueing system.

The Queueing Equation: We need the probability distribution of the Sojourn Time. The paper employs the Laplace Transform method to derive this.

Let $W^*(s)$ be the Laplace-Stieltjes transform of the waiting time distribution. For an M/G/1 queue (or M/M/1 as a simplified case), the transform of the total time (Waiting + Service) is given by the Pollaczek-Khinchine transform equation.

However, for the specific case of Exponential processing (μ) and Poisson arrivals (λ), the Completion Rate $F(t)$ (which is the CDF of the sojourn time) takes a closed form.

The Sojourn time T is the sum of the Waiting time W and the Service time X. For an M/M/1 queue, the probability density of the sojourn time is known to be $f_T(t) = (\mu - \lambda)e^{-(\mu-\lambda)t}$. We derive the Completion Rate CDF, $F(t)$, by integrating this density:

$$F(t \leq \tau) = \int_0^\tau (\mu - \lambda)e^{-(\mu-\lambda)t}\, dt$$

Let $u = (\mu - \lambda)t$, then $du = (\mu - \lambda)dt$:

$$F(t \leq \tau) = [-e^{-(\mu-\lambda)t}]_0^\tau = (-e^{-(\mu-\lambda)\tau}) - (-e^0) = 1 - e^{-(\mu-\lambda)\tau}$$

Subtracting the transmission delay t_{trans} to find the effective time τ_{eff}, we arrive at the final closed-form model used in our simulation:

Completion Probability Formula: The probability that a task with deadline τ is completed on time is given by:

$$F(t \leq \tau) = 1 - \exp\left(-(\mu - \lambda)\tau_{eff}\right) \tag{3}$$

Where:

- $\tau_{eff} = \tau - t_{trans}$: The effective time left for processing after subtracting transmission delay.
- μ: The service rate of the device.
- λ: The total arrival rate of tasks at that device (including local and offloaded tasks).
- Constraint: The system is stable only if $\mu > \lambda$ (Service rate exceeds arrival rate).

2.4 Probabilistic Assumptions

To make the mathematical model tractable, we make the following standard assumptions:

1. Independence of Service Times: The processing time of task k is statistically independent of task $k-1$.

2. Poisson Arrival Process: The arrival of tasks follows a Poisson process with rate λ. This implies that the inter-arrival times are exponentially distributed:

$$P(T_{inter} \leq t) = 1 - e^{-\lambda t}$$

3. Lossless Queue: We assume the buffer size is sufficiently large such that tasks are not dropped due to overflow, only due to deadline expiration.

4. Static Channel during Transmission: While processing is stochastic, the transmission time t_{trans} is treated as a constant for a given offloading event (simplified channel model) to focus the uncertainty analysis on the computational aspect.

Scribe Question 3: Probabilistic Reasoning and Dependencies

Explain how probabilistic relationships (e.g., dependence, independence, conditionality) are used within your project system to support reasoning, inference, or decision-making.

3.1 Conditional Probability and Inference

The core of the "Probabilistic Approach" is that decisions are made based on Conditional Expectations. A device does not simply ask "Is device j fast?"; it calculates the probability that a specific task will meet its deadline given the current state of the neighbor:

$$P(Y_{i,j}^k = 1 \mid \lambda_j) \tag{4}$$

This inference relies on the queueing dynamics derived in Section 2.3. The completion rate $F_{i,j}^k(\mathbf{a})$ is a rigorous conditional probability function:

$$F_{i,j}^k(\mathbf{a}) = P\left(W_j + X_j \leq \tau_k - t_{trans} \middle| \lambda_j = \sum_{n=1}^{N} a_{n,j}\lambda_n\right) \tag{5}$$

Here, the success of task k is conditioned on the aggregate arrival rate λ_j, which is a stochastic result of the strategy vector $\mathbf{a}$ across all N devices.

The term $\lambda_j = \sum_{n=1}^{N} a_{n,j}\lambda_n$ represents the total arrival rate at node j. The derivation of this condition follows the **Principle of Superposition** for Poisson processes. Since each offloading decision $a_{n,j}$ is an independent Bernoulli trial, the resulting traffic arriving at node j is the sum of N independent Poisson streams. This ensures that the aggregate arrival process is also Poisson, allowing us to maintain the stability constraint $\mu > \lambda_j$ in our decision-making logic.

3.1.1 Probabilistic Dependency Diagram

To visualize how local decisions propagate through the system to influence the final success event, we model the following dependency chain:

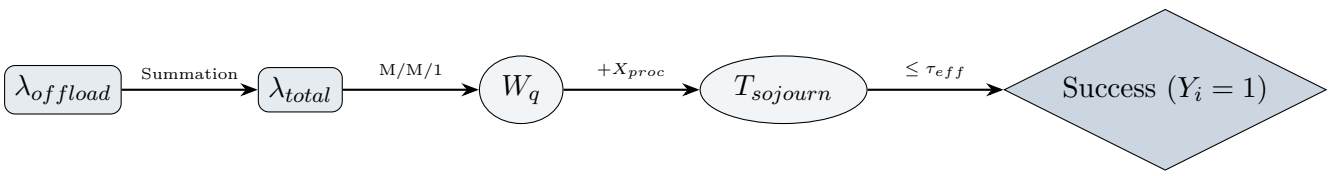

Figure 1: Probabilistic dependency chain showing the propagation of uncertainty from offloading decisions to task success.

3.2 Probabilistic Dependence: The Congestion Externality

The system models a complex web of dependencies between ostensibly independent devices.

Negative Dependence: There exists a negative probabilistic dependence between the tasks of different users.

- Let S_A be the event "User A's task succeeds".

- Let S_B be the event "User B's task succeeds".
- If User A and User B both offload to Device C, then $P(S_A \cap S_B) < P(S_A)P(S_B)$.

Why? Because User B's task increases the arrival rate λ_C. As seen in Eq 3, increasing λ decreases the exponential decay rate $(\mu - \lambda)$, thereby reducing the CDF value $F(t)$. This models the Congestion Externality: one user's optimization negatively impacts another user's probability of success.

3.3 Game Theoretic Formulation (Decision Making)

The project uses **Game Theory** to resolve these probabilistic dependencies. Since central optimization is NP-hard, we model the system as a non-cooperative game.

The Game Tuple: $G = \{N, A, u\}$

- N: Set of devices (players).
- A: Strategy space (offloading decisions $a_{i,j}$).
- u: Utility function (Expected Payoff).

The Expected Payoff (Objective Function): Each device i seeks to maximize its own utility U_i, which is defined as the weighted sum of completion probabilities for its tasks:

$$U_i(a_i, a_{-i}) = \sum_{k=1}^{K} w_k \cdot F_{i,j}^k(a_i, a_{-i}) \tag{6}$$

Note the notation (a_i, a_{-i}). This explicitly states that Device i's payoff depends on its own action a_i AND the actions of all other players a_{-i}.

3.4 Nash Equilibrium and Potential Functions

To prove that the system can reach a stable state (where probabilistic reasoning leads to a final decision), the paper utilizes the concept of a Potential Game.

We define a global Potential Function $\Phi(a)$ that captures the aggregate congestion in the network. The paper proves that if a device unilaterally changes its strategy to improve its own probabilistic utility U_i, the global potential $\Phi(a)$ strictly increases.

$$\Phi(a_{new}) - \Phi(a_{old}) > 0 \implies \text{System converges} \tag{7}$$

This ensures that despite the uncertainty and the complex dependencies, the decision-making process will converge to a Nash Equilibrium—a state where no device can improve its success probability by switching strategies.

Scribe Question 4: Model-Implementation Alignment

Describe how the current probabilistic model aligns with the project's implementation or experimental setup. Highlight any assumptions that influence design or evaluation choices.

4.1 Aligning the Math with Reality

At this stage, our probabilistic model is designed to act as the mathematical blueprint for our simulation code. In our experimental setup, we are simulating mobile devices that receive tasks and decide whether to process them locally or send them away.

To make our code reflect real-world unpredictability, we do not use fixed timers. Instead, our simulation aligns with the probabilistic model by using random number generators based on specific distributions:

- **Task Generators:** Our code generates new tasks using a Poisson process (rate λ).
- **Task Executors:** The simulated CPU processes tasks taking random amounts of time drawn from an Exponential distribution (rate μ).

This setup effectively builds a software version of an $M/M/1$ queue, allowing us to accurately measure the *Task Completion Rate* rather than just looking at the average speed.

4.2 Critical Understanding: Why Averages Lie (Jensen's Inequality)

In traditional models, systems are designed around the *average* execution time ($\mathbb{E}[T]$). However, we must look deeper into the risks hidden by averages. In mobile edge networks, the penalty for a task being late is rarely linear. The "cost" of delay is a *convex function*, where extreme delays (tail events) cause disproportionately high system failure rates.

We utilize **Jensen's Inequality** to prove mathematically why designing for the average time leads to system failure:

$$\mathbb{E}[C(T)] \geq C(\mathbb{E}[T]) \tag{8}$$

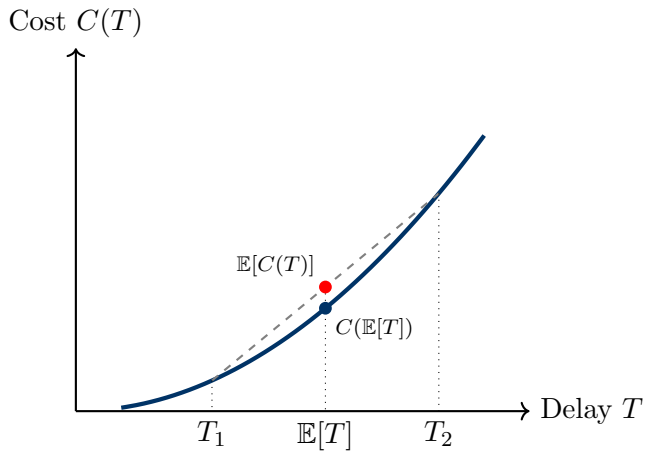

Figure 2: Geometric proof of Jensen's Inequality: The expected cost $\mathbb{E}[C(T)]$ on the secant line is strictly greater than the cost of the expected delay $C(\mathbb{E}[T])$ on the convex curve.

Where:

- T is the random task processing time.
- $C(T)$ is the convex cost/penalty function for delay.
- $\mathbb{E}[\dots]$ represents the expected (average) value.

We can prove why the average of the cost is higher than the cost of the average using a Taylor series expansion of $C(T)$ around the mean $\mu = \mathbb{E}[T]$:

$$C(T) \approx C(\mu) + C'(\mu)(T - \mu) + \frac{1}{2}C''(\mu)(T - \mu)^2$$

Taking the expectation on both sides:

$$\mathbb{E}[C(T)] \approx C(\mu) + C'(\mu)\mathbb{E}[T - \mu] + \frac{1}{2}C''(\mu)\mathbb{E}[(T - \mu)^2]$$

Since $\mathbb{E}[T - \mu] = 0$ and for a convex function the second derivative $C''(\mu) > 0$:

$$\mathbb{E}[C(T)] \approx C(\mathbb{E}[T]) + \frac{1}{2}C''(\mu) \cdot Var(T)$$

This proves that as long as there is variance in the network ($Var(T) > 0$), the actual system cost $\mathbb{E}[C(T)]$ will always be strictly greater than the cost calculated using fixed averages.

Simple Analogy: Imagine your teacher penalizes you for being late. Being 1 minute late costs 1 point, 2 minutes costs 4 points, and 3 minutes costs 9 points ($C(T) = T^2$). If you are 2 minutes late on average, the penalty of your average time is 4 points. But if you are unpredictable—on time one day (0 points) and 4 minutes late the next (16 points)—your *average penalty* is 8 points!

This proves that **the penalty of the average is strictly less than the average of the penalty**. Consequently, our implementation must simulate the full variance of the PDF to capture the true risk of system failure.

4.3 Current Assumptions Influencing Design

1. **Exponential Simplification:** We currently assume processing times follow an Exponential distribution. This simplifies our code to an $M/M/1$ queue, making early-stage bug tracking easier, even though real tasks might behave slightly differently.

2. **Instant Network Knowledge:** Our game theory implementation assumes all devices instantly know the workload (λ_j) of their neighbors. In the real world, broadcasting this takes time.

Scribe Question 5: Cross-Milestone Consistency and Change

Describe the current state of your project's probabilistic model. Identify the assumptions, components, or relationships that are presently well-defined, and indicate which aspects are expected to evolve or require refinement in subsequent milestones. Briefly justify your reasoning.

5.1 Progression from Milestone 1

In Milestone 1, our understanding was quite basic. We simply acknowledged that Task Processing Time (TPT) was a random variable $T \sim f_T(t)$ and that tasks arriving were unpredictable. We knew we had a problem (uncertainty), but we did not have the exact mathematical tools to solve it.

In this current stage (Milestone 2), our model has matured significantly. We have transitioned from abstract concepts to a concrete mathematical framework:

- **Well-Defined Queueing Dynamics:** We have formally defined the system using an $M/M/1$ queueing structure. We now have a closed-form mathematical equation (Equation 2) to calculate the exact probability of a task meeting its deadline.
- **Game Theoretic Logic:** We have successfully mapped the probabilistic behavior of the queue into a **Potential Game**, proving that devices making selfish probabilistic choices will eventually reach a stable Nash Equilibrium.

5.2 What Must Evolve in Future Milestones

While our core logic is now solid, some of our assumptions are "training wheels" that must be removed as we approach Milestones 3 and 4 to ensure real-world applicability.

1. **Moving Beyond Exponential (Heavy-Tailed Risks):** Right now, the Exponential distribution assumes that exceptionally long tasks are incredibly rare. However, the base paper notes that tasks like speech recognition follow Gamma or Gaussian distributions. In future milestones, we must evolve our $M/M/1$ model into a true $M/G/1$ model using Laplace transforms. We need to account for "heavy-tailed" distributions where extreme delays happen more often than the Exponential model predicts.
2. **Dynamic Parameter Calibration:** Currently, we assume the processing capability (μ) is a fixed parameter for each device. In reality, a mobile phone's CPU throttles when it gets hot. We will need to refine our model so that μ is a conditional variable that updates based on the device's thermal state and battery level.
3. **Stochastic Transmission Times:** We currently assume transmission time (t_{trans}) is fixed. To make our model fully robust, we must treat the network delay as its own random variable, meaning the total completion time will become the sum of two random variables (Waiting/Processing + Transmission).

Scribe Question 6: Open Issues and Responsibility Attribution

Identify any unresolved probabilistic questions or ambiguities in the project at this stage, and indicate which role(s) or tasks are responsible for addressing them in the next milestone.

6.1 Unresolved Probabilistic Ambiguities

As we develop a deeper critical understanding of our system, a few major ambiguities remain unresolved:

- **The "Stale Information" Problem:** Our game-theory model requires devices to know the arrival rate (λ) of peer devices to calculate conditional probability. But if a device broadcasts its λ, and 5 milliseconds later receives three new tasks, its broadcasted λ is now wrong. *How do we calculate the probability of success when the data we base our decisions on might be outdated?*
- **Correlated Task Arrivals:** We assume Poisson arrivals (independent random events). But in reality, if a user opens a complex webpage, a sudden "burst" of related tasks are generated at the exact same time. *How does our queueing model handle bursty, highly-correlated traffic?*

6.2 Responsibility Attribution for Milestone 3

To tackle these open issues and evolve our model, our five-member team will divide responsibilities as follows for the upcoming milestone:

1. **Scribe & Math Modeling Team (Urvil & Manthan):**
   - *Task:* Upgrade the mathematical model from $M/M/1$ to $M/G/1$.
   - *Goal:* Derive the Laplace transforms needed to support non-Exponential distributions (like Gamma) to handle the heavy-tail risks identified in Q5.
2. **Simulation & GitHub Team (Falak & Nikunj):**
   - *Task:* Begin writing the Python/C++ simulation based on the probabilistic rules defined in M2.
   - *Goal:* Create a test environment where we can intentionally inject "bursty" traffic to see how badly the Poisson assumption fails, addressing the ambiguity from Q6.
3. **Concept Map & Presentation Coordinator (Ishani):**
   - *Task:* Visually map the flow of conditional probability across the network.
   - *Goal:* Help the team and the audience visualize Jensen's Inequality and the compounding risks of the "Stale Information" problem in our D2D network.